

Phishing Simulation, Security Awareness Training and ML/DL-Based Phishing URL Detection

A Two-Part Cybersecurity Project Report

Part I: Simulated phishing campaigns (clone login page, malicious-attachment simulation, QR-code/OTP simulation) built for internal security awareness training.

Part II: A hybrid machine-learning / deep-learning pipeline for automated phishing URL detection.

Author: Ayush Gupta

Date: July 2026

All simulated phishing pages in this project display an explicit “this was a phishing simulation” disclosure after interaction and store no real credentials or personal data. They are intended solely for authorized, internal security awareness training.

Contents

I	Phishing Simulation and Security Awareness Training	5
1	Introduction and Vision	6
1.1	Motivation	6
1.2	Ethical and Operational Boundaries	6
2	Background: What is Phishing?	8
2.1	Definition	8
2.2	Common Categories of Phishing	8
2.3	Why Simulation-Based Training Works	9
3	Simulation 1: Clone Phishing – Fake Microsoft Login Page	10
3.1	Attack Scenario	10
3.2	Page Flow	11
3.3	Technical Implementation	12
4	Simulation 2: Malicious Attachment Disguised as a PDF (“HiApp”)	13
4.1	Attack Scenario	13
4.2	Behavior	13
4.3	Technical Implementation	14
5	Simulation 3: QR Code / OTP Phishing (“Claim Your Reward”)	15
5.1	Attack Scenario	15
5.2	Technical Implementation	16
6	Centralized Awareness Dashboard	17
6.1	Purpose	17
6.2	Architecture	17
6.3	Campaign Lifecycle	18
II	ML/DL-Based Phishing URL Detection	20
7	Motivation and Problem Statement	21
7.1	Limitations of the Original Approach	21
7.2	Design Goals for the New Pipeline	21
8	System Architecture	22
8.1	Overview	22
8.2	Branch A – XGBoost over Lexical/Structural Features	22

8.3	Branch B – Character-Level CNN over the Raw URL	23
8.4	Meta-Stacker – Combining Both Branches	23
8.5	Trusted-Domain Allow-List Safeguard	23
9	Dataset Construction	25
9.1	Live Data Sources	25
9.2	Synthetic Path Diversification	25
9.3	Final Dataset Composition	26
10	Training Methodology	27
10.1	Out-of-Fold Stacking Procedure	27
10.2	GPU Acceleration	27
11	Results	28
11.1	Held-Out Test Set Performance	28
11.2	Most Important Lexical Features	28
11.3	Adversarial and Sanity Test Suite	29
11.4	A Debugging Case Study: The Path-Complexity Bug	29
12	Web Application	31
12.1	Overview	31
12.2	Design Rationale	33
13	Conclusion and Future Work	34
13.1	Summary	34
13.2	Future Work	34

List of Figures

3.1	The pretext email as it lands in the target’s inbox: a spoofed sender name and address (<code>microsot_support@outlook.com</code> – note the misspelling), a fabricated password-expiration notice with a same-day deadline, and a <i>Reset Password</i> link. Gmail’s own spam filter already flagged it as suspicious, illustrating that even a fairly crude pretext can still reach and fool a recipient who does not stop to check the sender address or the filter’s warning.	10
3.2	Stage 1 – where the <i>Reset Password</i> link lands: the cloned Microsoft sign-in page, requesting an email/phone/Skype identifier. Visually matches the real Microsoft sign-in page’s layout, logo, and typography.	11
3.3	Stage 2 – after entering an identifier, the clone advances to a password prompt, echoing the identifier back exactly as the real flow does.	11
3.4	Stage 3 – immediately after “signing in,” the page reveals itself as a phishing simulation and lists the concrete red flags the target should check next time (address bar URL, unsolicited re-authentication requests, reporting channel). . .	12
4.1	HiApp’s disclosure screen, shown immediately on launch: it explains that the “PDF” was actually an executable, and that in a real attack this exact confusion is how malware gets installed.	14
5.1	The QR-code landing page: a “Claim Your Reward” prompt asking the target to enter a 6-digit OTP, styled to feel casual and low-stakes.	15
5.2	Immediately after submission, the page discloses the simulation and gives the specific rule the target should internalize: never enter a real OTP on a page reached via an unverified QR code.	16
6.1	The awareness dashboard, showing per-campaign totals (page views vs. the “dangerous” actions: passwords entered, OTPs entered, EXE launched) and a live recent-activity feed. Data shown here is illustrative sample data generated for this report.	18
6.2	End-to-end lifecycle of a single simulated phishing campaign, common to all three simulations in Part I.	19
8.1	Hybrid pipeline architecture: two independent branches (lexical-feature XG-Boost, and character-level CharCNN) are combined by a trained meta-stacker, with a final trusted-domain allow-list safeguard.	22
10.1	Training and stacking pipeline (<code>pipeline/train.py</code>), from raw data to held-out evaluation.	27
12.1	The Phishing URL Checker home page.	31

12.2 Result for a phishing test URL (<code>paypal-secure-login.tk</code>): 99.8% phishing risk, with the six triggered risk factors (suspicious TLD, missing HTTPS, hyphenated domain, high URL entropy, near-miss brand typo, credential/urgency keywords) and the individual XGBoost (96.4%) and CharCNN (99.9%) branch scores shown transparently.	32
12.3 Result for a legitimate URL (<code>wikipedia.org</code>): 99.76% safe, with no risk factors triggered and both branch scores low.	32

Part I

Phishing Simulation and Security Awareness Training

Chapter 1

Introduction and Vision

1.1 Motivation

Phishing remains the single most common initial-access technique used in cyberattacks: rather than breaking cryptography or exploiting a zero-day vulnerability, an attacker simply asks the victim to hand over credentials, install malware, or approve a fraudulent transaction — and dresses the request up as something routine and trustworthy. Because the weakest link is human judgement rather than a technical control, the most effective defense is not a firewall rule but *trained suspicion*: employees who have already seen what a convincing phishing attempt looks like, in a safe setting, are measurably less likely to fall for a real one.

This project has two complementary halves that both attack this same problem from opposite ends:

- **Part I – Offense-as-training.** A small set of realistic, self-contained phishing simulations (a cloned Microsoft login page, a disguised malicious attachment, and a QR-code/OTP reward scam) that can be sent to employees exactly as a real attacker would send them. Each simulation captures whether the target fell for it, and — critically — immediately reveals itself as a training exercise and explains what real-world red flags were present, turning a moment of embarrassment into a teachable moment.
- **Part II – Defense via automation.** A machine-learning system that classifies arbitrary URLs as phishing or legitimate in real time, so that even attempts nobody has been specifically trained against can be caught before a human has to make the judgement call at all.

1.2 Ethical and Operational Boundaries

Every simulation in this project is built around three non-negotiable rules:

1. **No real credentials or personal data are ever stored.** Login forms, OTP entry, and file execution are all purely client-side theater — nothing typed into them is transmitted to any credential store or third party.
2. **Immediate, unambiguous self-disclosure.** The instant a target completes the simulated action (submits a password, enters an OTP, opens the fake attachment), the page reveals itself as a security-awareness exercise and explains the real-world warning signs the target should have noticed.

3. **Authorized, internal use only.** These simulations are designed to be run by an organization against its own employees (or by an individual against consenting test subjects), analogous to commercial phishing-simulation products used by corporate security teams – never against the public or without consent.

Chapter 2

Background: What is Phishing?

2.1 Definition

Phishing is a social-engineering attack in which an adversary impersonates a trustworthy entity — a company, colleague, government agency, or well-known brand — in order to trick a victim into revealing sensitive information (passwords, OTPs, credit-card numbers), installing malicious software, or performing an action that benefits the attacker (e.g. wiring money). The term is a homophone of “fishing”: the attacker casts a baited hook (a convincing message) and waits for a victim to bite.

2.2 Common Categories of Phishing

Phishing is not a single technique but a family of related attacks that differ in delivery channel, targeting precision, and payload. The categories most relevant to this project are summarized below.

Type	Description
Email phishing	Mass, untargeted fraudulent emails impersonating a bank, vendor, or service, urging the recipient to click a link or open an attachment. The oldest and still most common phishing vector.
Spear phishing	A targeted variant aimed at a specific individual or organization, using personal details (name, role, recent activity) to increase credibility.
Whaling	Spear phishing aimed specifically at senior executives or other high-value targets, often impersonating a CEO or board member.
Clone phishing	An attacker copies a legitimate, previously-seen email or website almost exactly, replacing a link or attachment with a malicious one. <i>This project’s Microsoft login clone is a clone-phishing simulation.</i>
Smishing	Phishing delivered via SMS text message, typically containing a shortened or disguised link.
Vishing	Voice phishing conducted over a phone call, often impersonating tech support, a bank’s fraud department, or a government agency.
Quishing (QR-code phishing)	A malicious URL is embedded in a QR code rather than a clickable link, exploiting the fact that a scanned QR code’s destination is not visible before the phone’s browser opens it. <i>This project’s QR/OTP reward simulation is a quishing simulation.</i>

Type		Description
Malicious-attachment phishing		The email's payload is a disguised executable or macro-laden document rather than a credential-harvesting link; opening it installs malware directly. <i>This project's "HiApp" fake-PDF simulation demonstrates this technique.</i>
Pharming		DNS or hosts-file manipulation redirects a victim from a legitimate domain to a fraudulent one even when they type the correct address.
Business Email Compromise (BEC)		An attacker compromises or spoofs a real corporate email account to authorize fraudulent wire transfers or data disclosure, typically without any malicious link at all.

2.3 Why Simulation-Based Training Works

Studies of enterprise security programs consistently find that periodic, realistic phishing simulations — paired with immediate, specific feedback at the moment of failure — reduce click-through rates on subsequent real phishing attempts far more effectively than annual slideshow training alone. The mechanism is simple: reading about phishing is abstract, but nearly entering your own password into a fake Microsoft login page and then being told exactly which visual and technical cues gave it away creates a memorable, specific mental model of what to look for next time. This is the design principle behind every simulation in Part I of this project.

Chapter 3

Simulation 1: Clone Phishing – Fake Microsoft Login Page

3.1 Attack Scenario

The simulated campaign begins with an email sent to the target employee, styled as an automated IT notice warning that their Microsoft 365 password is about to expire and urging them to click a *Reset Password* link before a same-day deadline – a deliberately mild urgency cue (a looming deadline, framed as routine IT housekeeping) chosen because it is far more effective at suppressing suspicion than an alarming one.

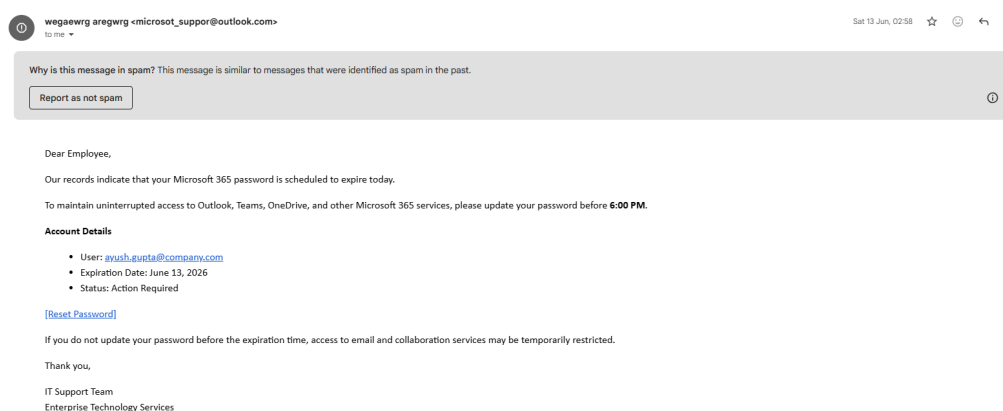


Figure 3.1: The pretext email as it lands in the target’s inbox: a spoofed sender name and address (`microsot_support@outlook.com` – note the misspelling), a fabricated password-expiration notice with a same-day deadline, and a *Reset Password* link. Gmail’s own spam filter already flagged it as suspicious, illustrating that even a fairly crude pretext can still reach and fool a recipient who does not stop to check the sender address or the filter’s warning.

The *Reset Password* link is the payload: it points not to a real Microsoft domain but to the locally-hosted clone of the Microsoft/Outlook sign-in page described below. Because the destination page is visually indistinguishable from the real thing, an employee who clicks through without checking the address bar will often proceed straight through the sign-in flow exactly as they would on the genuine site.

3.2 Page Flow

The clone reproduces Microsoft’s real two-step sign-in flow (email, then password) as a single-page application with three view states, and only reveals the training disclosure after both steps are completed – mirroring the real attack sequence a credential-harvesting kit would use.

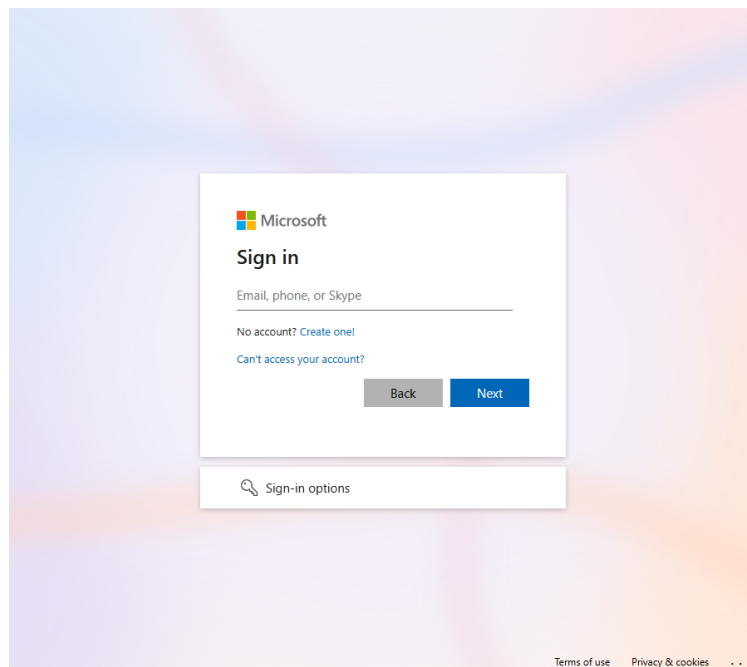


Figure 3.2: Stage 1 – where the *Reset Password* link lands: the cloned Microsoft sign-in page, requesting an email/phone/Skype identifier. Visually matches the real Microsoft sign-in page’s layout, logo, and typography.

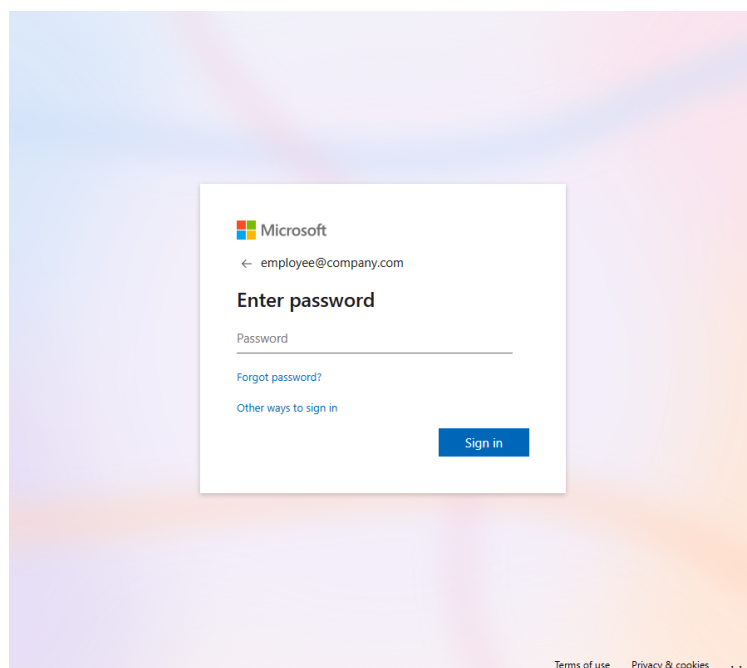


Figure 3.3: Stage 2 – after entering an identifier, the clone advances to a password prompt, echoing the identifier back exactly as the real flow does.

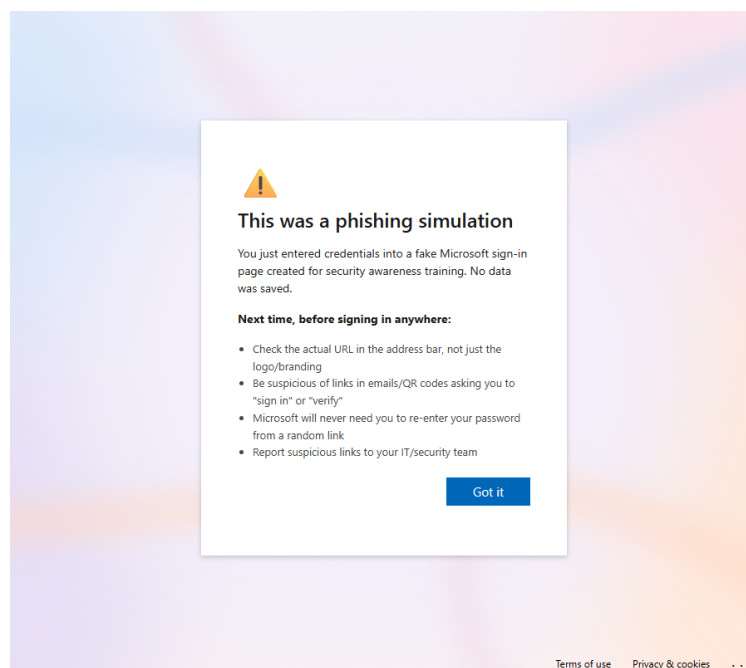


Figure 3.4: Stage 3 – immediately after “signing in,” the page reveals itself as a phishing simulation and lists the concrete red flags the target should check next time (address bar URL, unsolicited re-authentication requests, reporting channel).

3.3 Technical Implementation

- **Stack:** static HTML/CSS/JavaScript, no backend dependency for the page itself.
- **State machine:** three `<section>` blocks (`section_username`, `section_pwd`, `section_final`) toggled via a `d-none` CSS class, driven by `app.js`.
- **No credential capture:** the password field’s value is read only to gate the transition to the disclosure screen – it is never transmitted or written to storage.
- **Telemetry:** on page load and on credential submission, a fire-and-forget `fetch()` call posts an anonymous event (`page_view` / `credential_submit`) to the central tracking dashboard described in Chapter 5, tagged with campaign id `ms-login`.

Chapter 4

Simulation 2: Malicious Attachment Disguised as a PDF (“HiApp”)

4.1 Attack Scenario

The pretext email reads:

“Please review the attached document urgently and confirm before end of day.”

The attachment appears to be a PDF, but is in fact a packaged executable (**HiApp.exe**, built with Electron). This mirrors one of the oldest and still most effective malware-delivery techniques: disguising an executable’s true file type using a deceptive filename, icon, or double extension so that a rushed employee opens it without checking the real extension.

4.2 Behavior

On launch, the program immediately displays a full-screen disclosure – there is no hidden payload beyond this message, since the program’s sole purpose is the training exercise itself.

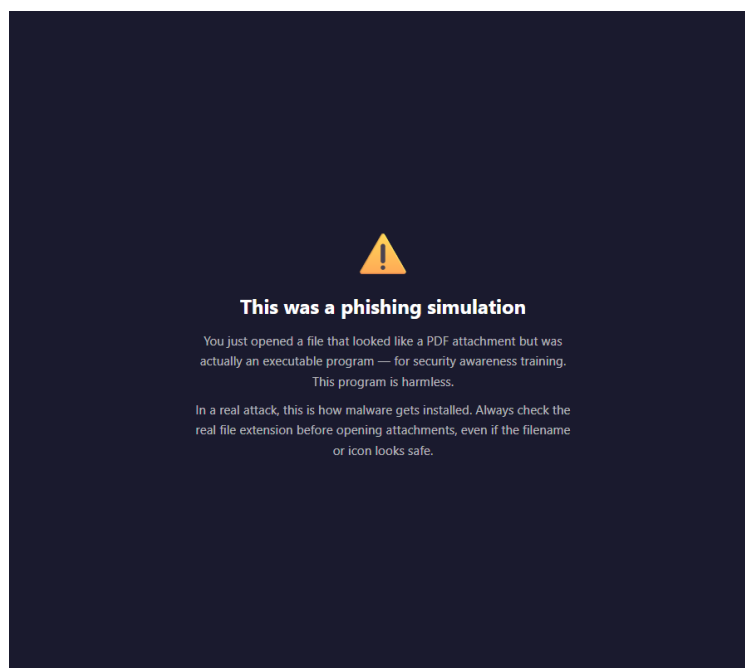


Figure 4.1: HiApp’s disclosure screen, shown immediately on launch: it explains that the “PDF” was actually an executable, and that in a real attack this exact confusion is how malware gets installed.

4.3 Technical Implementation

- **Stack:** Electron (`main.js` creates a single `BrowserWindow` loading a local `index.html`), packaged for Windows via `electron-builder` with a one-click NSIS installer profile.
- **Deceptive delivery (simulated):** in a real campaign this binary would be renamed/iconed to resemble a PDF attachment; the underlying Electron app itself is harmless and performs no filesystem or network action beyond the training disclosure.
- **Telemetry:** on launch, an `exe_run` event is posted to the dashboard under campaign id `exe`.

Chapter 5

Simulation 3: QR Code / OTP Phishing (“Claim Your Reward”)

5.1 Attack Scenario

The target receives a QR code (e.g. printed on a flyer, or emailed as an image) captioned “Scan to claim your reward.” Scanning it with a phone camera opens a mobile-styled page asking for a 6-digit OTP. This simulates a real and increasingly common attack pattern (*quishing*): the attacker relies on the fact that a scanned QR code’s destination URL is largely invisible to the user before their browser navigates to it, and pairs it with the classic pretext of an unexpected reward to short-circuit suspicion. In a real attack this OTP would be the one-time code from the victim’s actual bank or email account, harvested in real time to complete an account-takeover elsewhere.

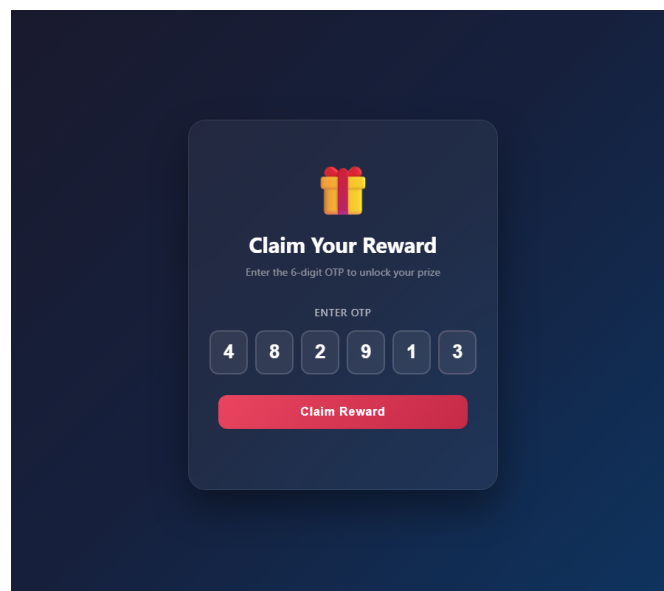


Figure 5.1: The QR-code landing page: a “Claim Your Reward” prompt asking the target to enter a 6-digit OTP, styled to feel casual and low-stakes.

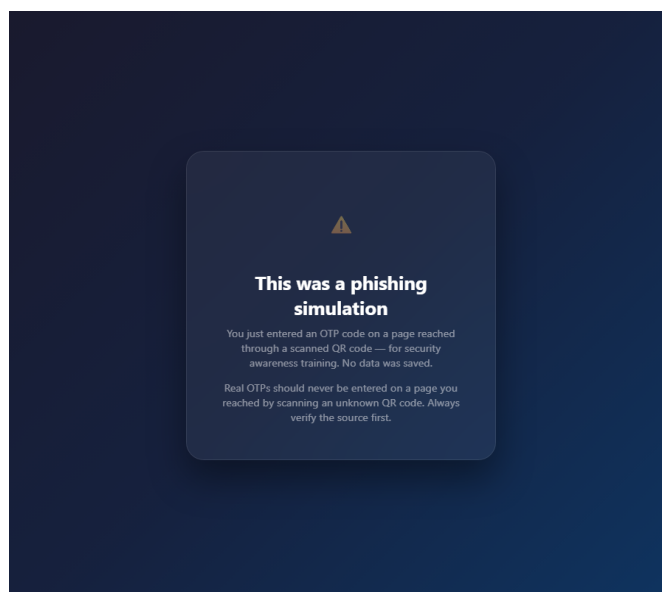


Figure 5.2: Immediately after submission, the page discloses the simulation and gives the specific rule the target should internalize: never enter a real OTP on a page reached via an unverified QR code.

5.2 Technical Implementation

- **Stack:** static HTML/CSS/JavaScript, with auto-advancing individual OTP digit inputs (focus moves to the next box as each digit is typed, matching the UX of real OTP-entry pages so the simulation feels authentic).
- **No OTP capture:** the entered digits are never transmitted or stored; they are read only to trigger the transition to the disclosure view.
- **Telemetry:** `page_view` and `otp_submit` events are posted under campaign id `qr-otp`.

Chapter 6

Centralized Awareness Dashboard

6.1 Purpose

All three simulations report anonymous, aggregate telemetry to a single dashboard, so that whoever runs the awareness campaign can measure — across an entire organization or test group — how many people viewed each lure, and how many actually fell for it (entered a password, entered an OTP, or ran the fake executable). This is the same measurement loop used by commercial phishing-simulation platforms: without it, a training exercise cannot demonstrate its own effectiveness.

6.2 Architecture

- **Stack:** Node.js + Express, with events persisted to a flat `events.json` file (sufficient for a single-organization awareness exercise; the schema is a simple append-only event log).
- **Endpoints:**
 - `POST /api/event` – records a `{campaign, type}` event along with timestamp, source IP, and user agent.
 - `GET /api/stats` – returns aggregate per-campaign counts and the 25 most recent events, polled by the dashboard every 4 seconds.
 - `POST /api/reset` – clears all recorded events, gated behind a shared secret key.
- **Validation:** both campaign id and event type are checked against a fixed allow-list server-side, so malformed or spoofed events cannot pollute the dashboard.

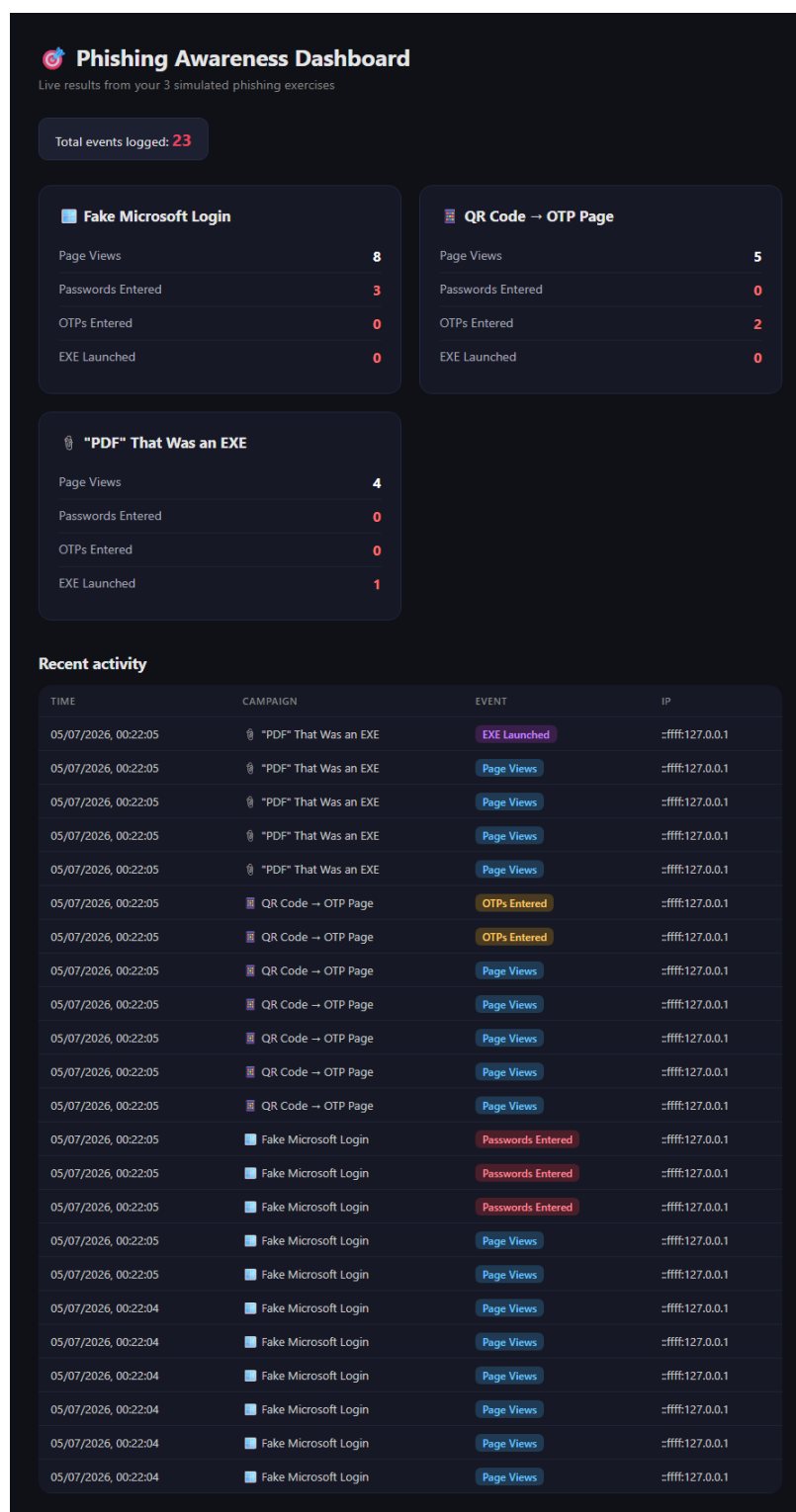


Figure 6.1: The awareness dashboard, showing per-campaign totals (page views vs. the “dangerous” actions: passwords entered, OTPs entered, EXE launched) and a live recent-activity feed. Data shown here is illustrative sample data generated for this report.

6.3 Campaign Lifecycle

Figure 6.2 summarizes the end-to-end lifecycle common to all three simulations: an attacker-styled pretext delivered by email or QR code, a convincing but inert simulated payload, an

immediate self-disclosure at the moment of “compromise,” and anonymous telemetry feeding the aggregate dashboard.

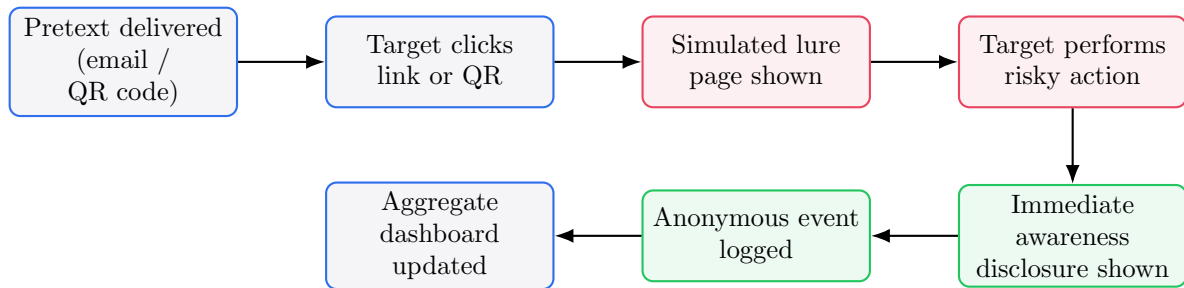


Figure 6.2: End-to-end lifecycle of a single simulated phishing campaign, common to all three simulations in Part I.

Part II

ML/DL-Based Phishing URL Detection

Chapter 7

Motivation and Problem Statement

7.1 Limitations of the Original Approach

The project began with a conventional phishing-URL classifier: a single scikit-learn model (RandomForest / GradientBoosting / XGBoost, model-selected by cross-validated ROC-AUC) trained on a public 30-feature dataset in which most features required a *live* lookup at prediction time — a WHOIS query for domain age, a screen-scraped Google search to check indexing, and calls to two third-party ranking services (the Alexa traffic-rank API and `checkpagerank.net`). In practice this made the classifier fragile in three concrete ways:

- **Dead dependencies.** The Alexa API was discontinued in 2022, and `checkpagerank.net` no longer serves the expected response — both features silently degrade to a constant value for every URL, contributing noise rather than signal.
- **Fragile dependencies.** WHOIS lookups and Google-search scraping are rate-limited and frequently blocked outright, meaning classification could fail or time out on exactly the live traffic a production system needs to handle.
- **Latency.** Because every prediction required several live network round-trips, a single classification could take seconds — unacceptable for inline, real-time URL screening.

7.2 Design Goals for the New Pipeline

The rebuilt pipeline was designed around four goals:

1. **Fully offline inference.** Every feature used at prediction time must be computable from the URL string alone, in microseconds, with no external network call.
2. **Combine complementary signal sources.** Hand-engineered features encode domain expertise (what a security analyst would look for) but cannot anticipate every future obfuscation trick; a learned representation over raw characters can pick up patterns no one thought to encode by hand. Using both, combined by a trained meta-model rather than a fixed rule, should outperform either alone.
3. **Train and evaluate on live, current data** rather than a static, multi-year-old public dataset whose phishing examples no longer reflect current attacker tooling.
4. **Guard against the model's own blind spots** with an explicit, auditable safeguard rather than relying solely on statistical generalization.

Chapter 8

System Architecture

8.1 Overview

The final system is a **hybrid ensemble** of two independently-trained classifiers, combined by a logistic-regression meta-model, with a trusted-domain allow-list applied as a final safeguard. Figure 8.1 shows the full inference-time data flow.

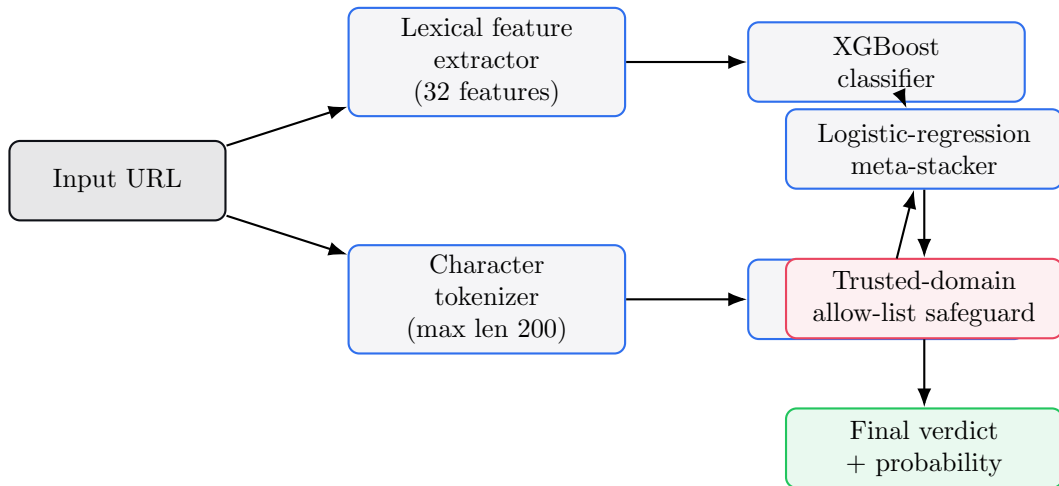


Figure 8.1: Hybrid pipeline architecture: two independent branches (lexical-feature XGBoost, and character-level CharCNN) are combined by a trained meta-stacker, with a final trusted-domain allow-list safeguard.

8.2 Branch A – XGBoost over Lexical/Structural Features

This branch computes 32 numeric features from the URL string alone (implemented in `pipeline/lexical_features.py`), grouped into four families:

- **Length/composition statistics:** URL, hostname, path and query length; counts of dots, hyphens, underscores, slashes, digits and special characters; digit-to-length ratio.
- **Randomness/obfuscation indicators:** Shannon entropy of the full URL and of the host-name alone (machine-generated or algorithmically-obfuscated domains tend to look statistically “noisier” than human-chosen names); presence of a raw IP-address host; presence of an @ symbol; suspicious // mid-path redirection; punycode (xn--) domains, which can encode homograph/look-alike Unicode characters.

- **Brand-impersonation detection:** domain tokens (split on hyphens/ digits) are compared via Levenshtein edit distance against a curated list of frequently-impersonated brand names (PayPal, Microsoft, Apple, major banks, etc.); a near-miss token that is *not* an exact match to the real brand domain (e.g. the token `paypa1` in `paypa1-secure-login.tk`) is a strong typosquatting signal. A separate feature flags a brand name appearing in a subdomain while absent from the actual registered domain.
- **Structural/contextual flags:** known-abused top-level domains (`.tk`, `.xyz`, `.top`, etc.), known URL-shortener domains, HTTPS usage, non-standard ports, subdomain count/depth, path depth, query parameter count, and a count of credential/urgency keywords (`login`, `verify`, `secure`, `suspend`, ...) appearing anywhere in the URL.

These features are passed to a gradient-boosted tree ensemble (`XGBClassifier`, 300 trees, max depth 6, learning rate 0.08), with `scale_pos_weight` set to the training set’s negative/positive class ratio to correct for class imbalance.

8.3 Branch B – Character-Level CNN over the Raw URL

Rather than relying solely on hand-designed features, the second branch (`pipeline/char_model.py`) learns directly from the URL’s raw character sequence, so that sub-word patterns no engineered feature explicitly captures — unusual character-substitution patterns, novel token structures, or unseen typosquat variants — can still be picked up.

- Each URL is lower-cased and encoded as a fixed-length sequence of integer character ids (vocabulary of 61 characters covering letters, digits, and common URL punctuation; sequences truncated/padded to 200 characters).
- A learned character embedding (dimension 32) feeds three parallel 1-D convolutions with kernel sizes 3, 5, and 7 (64 filters each), capturing character n-gram patterns at multiple scales.
- Global max-pooling over each convolution’s output is concatenated and passed through two fully-connected layers (64 units, then a single logit), with dropout (0.3) for regularization.
- Trained with the Adam optimizer and binary cross-entropy loss, on an NVIDIA GPU (CUDA) for fast iteration — full training on this dataset converges in well under two minutes on a consumer-grade GPU, versus multiple times longer on CPU alone.

8.4 Meta-Stacker – Combining Both Branches

Rather than averaging the two branches’ predictions with a fixed weight, a logistic-regression meta-model is trained on their *out-of-fold* predicted probabilities (Figure 10.1), so the combination weight for each branch is learned from data rather than guessed. This lets the ensemble automatically defer to whichever branch is more reliable for a given region of URL-space, rather than applying one static blend everywhere.

8.5 Trusted-Domain Allow-List Safeguard

No amount of synthetic training-path diversity can cover every real-world URL convention. During development, the CharCNN branch was found to occasionally flag well-known, entirely legitimate URLs with unusual but common path shapes — for example, code-hosting “organization/repository” paths such as `github.com/anthropics/claude-code` — because such path

conventions were under-represented relative to the sheer diversity of real-world phishing paths seen during training.

Rather than chasing ever more synthetic path variety indefinitely, the pipeline adds an explicit, auditable safeguard used by production anti-phishing systems in practice: a curated allow-list of roughly 8,400 registered domains drawn from the top 50,000 entries of the Cisco Umbrella popularity list. At inference time, if a URL’s *registered domain* (parsed correctly via public-suffix rules, not a naive substring match) is on this list *and* the URL shows none of the hard risk markers (no raw-IP host, no punycode, no @ symbol, and HTTPS is used), the ensemble’s probability is capped at a low value. Because the match is on the properly-parsed registered domain, an attacker cannot bypass it by embedding a trusted name as a subdomain of their own domain (e.g. `github.com.verify-login.tk` does *not* match `github.com`) — this was explicitly verified in the adversarial test suite (Section 9.3).

Chapter 9

Dataset Construction

9.1 Live Data Sources

Rather than relying on a static, multi-year-old public dataset, the pipeline builds its training data from two live, continuously-updated public feeds ([pipeline/build_dataset.py](#)):

- **Phishing examples:** the PhishTank verified-online feed — 65,302 unique, human-verified, currently-online phishing URLs at data-collection time.
- **Legitimate examples:** the Cisco Umbrella Popularity List, a continuously-updated ranking of the most-queried domains on the public internet.

9.2 Synthetic Path Diversification

The Umbrella list provides only bare domains (e.g. `github.com`), not full URLs with realistic paths. An early version of this pipeline generated legitimate examples using only the domain root plus a small fixed set of generic paths (`/login`, `/about`, ...). This produced a subtle but important bug: because real phishing URLs are, on average, structurally messier than that narrow template, the model learned to associate *any* sufficiently complex path with phishing — a spurious correlation, not a real signal, and one that produced false positives on legitimate URLs with realistic paths (see Section 9.4).

The dataset builder was corrected to generate much richer, realistic legitimate URLs per domain: randomized subdomains (`www`, `blog`, `shop`, `docs`, `api`, ...), multi-segment hyphenated slugs drawn from a 100-word vocabulary (mimicking blog-post and product-page URLs), numeric and hexadecimal identifiers (mimicking article/product/session ids), realistic query strings (`utm_source`, `ref`, `id`, ...), and an explicit “namespace/resource” path pattern (mimicking code-hosting and package-registry URL conventions such as `/org/repo`). The most-popular 30,000 domains are additionally sampled twice, so that heavily-trafficked real-world domains get richer path coverage rather than a single random guess.

9.3 Final Dataset Composition

Class	Count
Phishing (label = 1)	65,302
Legitimate (label = 0)	119,778
Total	185,080

Table 9.1: Final training dataset composition.

An 85/15 stratified split reserves 27,762 URLs as a held-out test set, never seen during any stage of training or hyperparameter/threshold selection.

Chapter 10

Training Methodology

10.1 Out-of-Fold Stacking Procedure

To train the meta-stacker without leaking information from a branch’s own training fold into its stacking input, both branches are trained using 5-fold stratified cross-validation on the training split, with each fold’s held-out predictions assembled into a full set of out-of-fold probabilities. The meta-stacker is trained on these out-of-fold probabilities only. Both branches are then refit on the *entire* training split for production use and for final held-out evaluation.

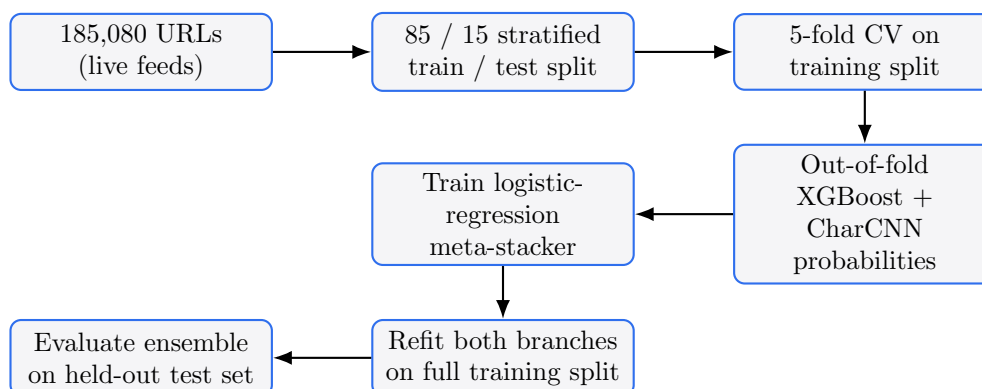


Figure 10.1: Training and stacking pipeline ([pipeline/train.py](#)), from raw data to held-out evaluation.

10.2 GPU Acceleration

The CharCNN branch is trained on an NVIDIA GPU (CUDA 12.4 build of PyTorch). Moving training from CPU to GPU was necessary purely for iteration speed: on this dataset size, GPU training reduced full pipeline training time (feature extraction + 5-fold CV for both branches + final refits) from an estimated 10–15 minutes on CPU to under 5 minutes end-to-end.

Chapter 11

Results

11.1 Held-Out Test Set Performance

Table 11.1 reports accuracy and ROC-AUC for each branch individually and for the final stacked ensemble, all measured on the 27,762-URL held-out test set that took no part in training, cross-validation, or threshold selection.

Model	Accuracy	ROC-AUC
XGBoost (lexical features only)	97.71%	0.9976
CharCNN (raw URL only)	99.13%	0.9997
Hybrid ensemble (stacked)	99.30%	0.9996

Table 11.1: Held-out test set performance, 27,762 URLs.

The stacked ensemble outperforms either branch alone on accuracy, confirming that the two branches make partially independent errors that the meta-stacker can correct for.

11.2 Most Important Lexical Features

Table 11.2 lists the ten most important features for the XGBoost branch, by Gini importance. Subdomain count dominates, consistent with the well-established pattern of phishing kits using long, multi-level subdomains (often on compromised or free-hosting infrastructure) to bury the true domain from a casual glance at the URL.

Feature	Importance
Number of subdomains	0.333
Longest path/word segment length	0.080
Path length	0.060
Suspicious TLD flag	0.058
HTTPS usage	0.043
Path depth	0.039
Hostname length	0.035
Number of hyphens	0.033
Presence of @ symbol	0.028
Number of dots	0.026

Table 11.2: Top-10 lexical features by XGBoost Gini importance.

11.3 Adversarial and Sanity Test Suite

Beyond the held-out statistical test set, a hand-built suite of 14 URLs was used to sanity-check behavior on cases a random held-out sample might not emphasize — well known legitimate sites with unusual path conventions, obvious phishing patterns, and one adversarial look-alike-domain construction. The final pipeline (including the allow-list safeguard) classified all 14 correctly, summarized in Table 11.3.

URL	Expected	Predicted
google.com	Legit	Legit
github.com/anthropics/claude-code	Legit	Legit
gitlab.com/gitlab-org/gitlab	Legit	Legit
en.wikipedia.org/wiki/Phishing	Legit	Legit
youtube.com/watch?v=...	Legit	Legit
stackoverflow.com/questions/...	Legit	Legit
amazon.com/dp/...	Legit	Legit
npmjs.com/package/react	Legit	Legit
paypal-secure-login.tk/verify/account	Phishing	Phishing
192.168.1.1/login.php	Phishing	Phishing
coincommunity-ta.netlify.app	Phishing	Phishing
microsoft-support-verify.xyz/...	Phishing	Phishing
secure-appleid-verify.com/signin	Phishing	Phishing
github.com.verify-login.tk/... (adversarial)	Phishing	Phishing

Table 11.3: Hand-built sanity/adversarial test suite – 14/14 correct on the final pipeline.

The last row is the most important: it embeds the trusted string `github.com` as a *subdomain* of an attacker-controlled domain (`verify-login.tk`). The allow-list correctly does *not* fire here, because it matches on the properly-parsed registered domain rather than a substring search – confirming the safeguard cannot be trivially bypassed.

11.4 A Debugging Case Study: The Path-Complexity Bug

The first fully-trained version of this pipeline reached 99.02% held-out accuracy, yet flagged <https://github.com/anthropics/claude-code> as 99.66% likely to be phishing. Investigating the individual lexical features for that URL showed nothing suspicious (HTTPS used, no IP

host, no suspicious TLD, no brand typosquatting) – yet the XGBoost branch alone assigned it a phishing probability of 1.0. The root cause, traced by inspecting the training data generator, was that the synthetic legitimate URLs used only about 20 fixed, trivially simple path templates, while real phishing URLs are naturally far more path-diverse; the model had learned “complex path” as a proxy for phishing rather than any of the intended features. Section 7.2 describes the fix (richer, realistic synthetic path generation), and Section 6.4 describes the complementary allow-list safeguard added once it became clear that no amount of synthetic diversity alone could realistically cover every legitimate path convention on the internet. This episode is included here deliberately: it is a useful illustration of why held-out accuracy alone is not sufficient validation, and why targeted, hypothesis-driven spot-checks against real-world URLs remain an essential part of evaluating any URL classifier.

Chapter 12

Web Application

12.1 Overview

The trained pipeline is served through a Flask web application (`app.py`) exposing a single page: a URL input box and, on submission, a verdict panel showing the overall phishing probability, the detected risk factors (in plain language, not raw feature names), and a transparent breakdown of each branch's individual score.

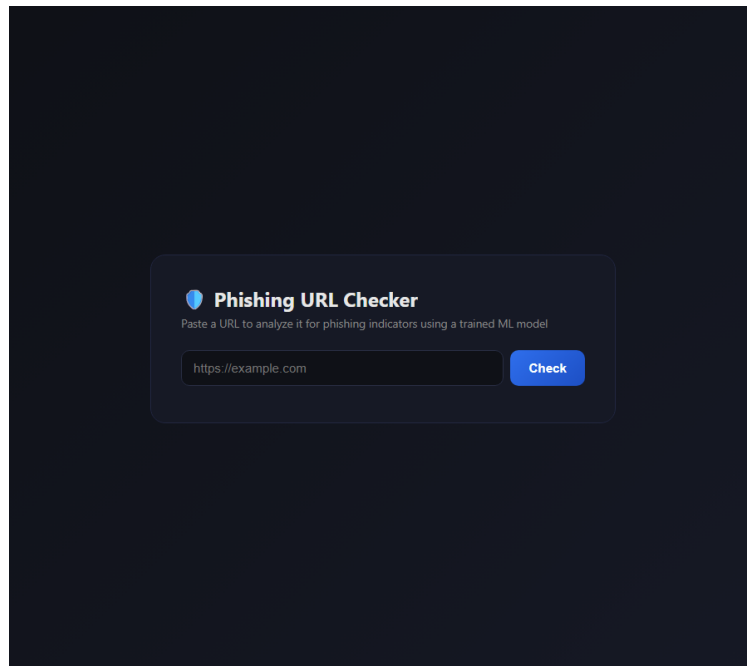


Figure 12.1: The Phishing URL Checker home page.

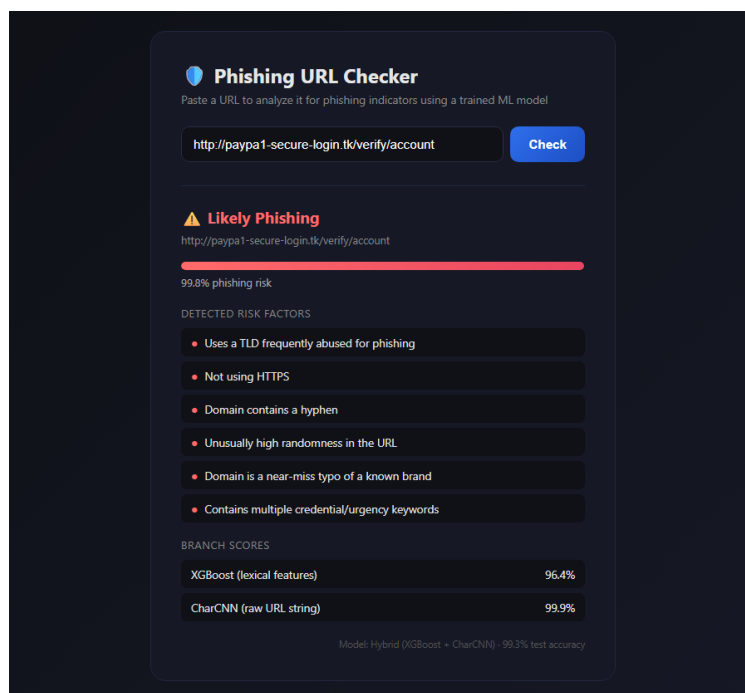


Figure 12.2: Result for a phishing test URL (`paypal1-secure-login.tk`): 99.8% phishing risk, with the six triggered risk factors (suspicious TLD, missing HTTPS, hyphenated domain, high URL entropy, near-miss brand typo, credential/urgency keywords) and the individual XGBoost (96.4%) and CharCNN (99.9%) branch scores shown transparently.

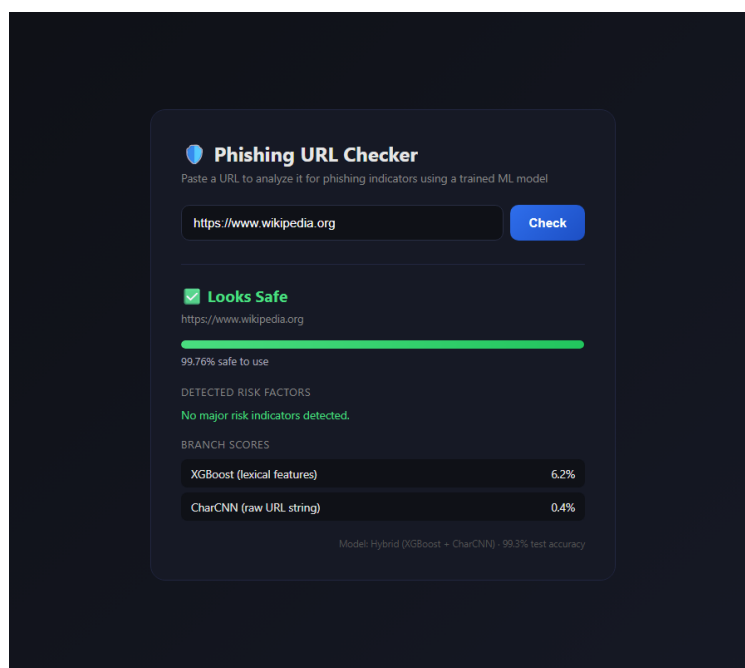


Figure 12.3: Result for a legitimate URL (`wikipedia.org`): 99.76% safe, with no risk factors triggered and both branch scores low.

12.2 Design Rationale

Showing the two branch scores separately (rather than only the final blended verdict) was a deliberate transparency choice: a security analyst reviewing a borderline case can see *why* the model reached its conclusion – whether the structural features or the raw-character model drove the decision – rather than being handed an opaque single number.

Chapter 13

Conclusion and Future Work

13.1 Summary

This project delivers two complementary, independently useful systems: a set of realistic, self-contained phishing simulations with built-in measurement and immediate self-disclosure, suitable for running internal security-awareness exercises; and a hybrid ML/DL phishing-URL classifier that improves on a single-model baseline by combining hand-engineered lexical features with a learned character-level representation, trained on live rather than static data, and hardened with an explicit, auditable allow-list safeguard. The final system reaches 99.3% accuracy (ROC-AUC 0.9996) on a held-out test set of 27,762 URLs never seen during training, and passes a 14-case hand-built adversarial sanity suite including a subdomain-impersonation attack against its own safeguard.

13.2 Future Work

- Extend the simulation suite with additional pretexts (spear-phishing using scraped public employee data, vishing call scripts) and multi-organization campaign management in the dashboard.
- Periodically retrain the detection pipeline on refreshed PhishTank/Umbrella snapshots to track drifting attacker tooling over time, and monitor for distributional drift in production.
- Explore replacing the fixed 100-word synthetic-path vocabulary with paths sampled from a real, licensed web crawl, to further close the residual gap between synthetic and real-world legitimate URL diversity.
- Add explainability tooling (e.g. SHAP values) for the XGBoost branch to give security analysts a per-prediction feature attribution, beyond the current static global importance ranking.